

Project Report

Task 2 - Testing & Evaluation

Introduction

This report documents task 2 testing and evaluation of the Gomoku game. The program was first developed in task 1 as a terminal-based game. In this task 2, different tests will be conducted to see whether the program run as expected, and also collect feedback from different users. Then, we will fix bugs in this task if found, and new features may be added to improve game experience based on feedback and testing.

Pros and Cons of the Program Design

Strengths

1. Modular function design - Different logic is separated into functions, like printing the game board (`board_print`), win check (`check_win_v2`) etc. By this modular design, each function is reusable, making the code clearer and easier to maintain, also testing.
2. Comprehensive input validation - The program will first check whether the player's input is empty or non-numeric (For coordinate input). Next, the program will check whether the player has selected an occupied cell or selected out-of-range coordinates. The player will be asked to input again until the data is valid after validation.
3. Support PvP and PvM mode - Player can choose their preferred game mode at the start, either PvM (Player vs Machine) or PvP (Player vs Player). If PvM mode is chosen, the machine player will prevent human player from winning and trying to win itself, and finally a random fallback. If PvP mode is chosen, the program will show the current player, and the two players will be asked to choose a cell one by one until one of them wins or the board is full.
4. Clear visual feedback - Terminal output is colour-coded (`termcolor`) and the game board is formatted (`tabulate` with `rounded_grid` style), both of them improve the user experience.

Weaknesses

1. PvM machine only searches one move deep - The program will only inspect nearby cells of the last human placed piece simply. It does not evaluate multiple future moves or find another cell that have the highest possibility to win within the whole board, resulting in a weak opponent that can be easily beaten by an human player.
2. Interface could be improved - The program is only supported in terminal view, including game board, inputting coordinates all have to be in terminal. As screen-cleaning reliance on ANSI escape codes, if the platform do not support it, it may not render correctly. Also, further development are limited due to limited information can show in terminal, like graphic user interface might be a possible solution to show more information and provide better game experience.
3. Only one round is supported - The program will end after one round, the player cannot choose to start a new round, instead they have to run the code again manually.

Test Data and Test Cases

Test Environment: MacOS Python 3.x

Input Validation

Input validation is a important part as player interact with game through terminal input and output. If the player accidentally submitted invalid data to the program, the program may not able to handle it (including empty value, out-of-range value, wrong data type, etc.), resulting the game stop unexpectedly and user need to restart it. To avoid that happened, we need input validation to detect invalid data, and let the player entered again.

The program accept input in **two places**, including:

1. **Game Mode selection** – User have to select game mode before starting the game by typing "1" (PvP) or "2" (PvM).
2. **Coordinate input** – Players need to input the coordinate of the cell to place their piece by entering row number and column number.

Each test uses differnt tyoe of value, including valid data that we expected player to input, empty value, not within the game board size range and occupied cell. Expect valid data that the program can process further, other data should be labeled invaild and reject to ask for input again.

1. Game Mode Selection

At the start of this game, the program will ask player which mode they want play. Player are allowed to choose PvM or PvP by entering 1 and 2, other input are not accepted.

Test ID	Input	Data Category	Expected	Actual
IV-M-1	"1"	Valid	Print "You have selected PvP mode" and PvP mode entered	Passed (As expected)
IV-M-2	"2"	Valid	Print "You have selected PvM mode", and PvM mode entered	Passed (As expected)
IV-M-3	"3"	Invalid	Print "Invalid input", and then ask to input again	Passed (As expected)
IV-M-4	"0"	Invalid	Print "Invalid input", and then ask to input again	Passed (As expected)
IV-M-5	"abc"	Unmatched data type	Print "Invalid input", and then ask to input again	Passed (As expected)
IV-M-6	"" (empty value)	Empty	Print "Invalid input", and then ask to input again	Passed (As expected)
IV-M-7	" 1 " (with spaces)	Whitespace	Print "Invalid input", and then ask to input again	Passed (As expected)

2. Coordinate Input

The inputted coordinate will be validate by three steps:

1. **Empty check** – If the player entered nothing ("") , the program will show "You type nothing!!! Try again, please.". The player will be asked to entered again if this check is not passed.
2. **Numeric check** – As coordinates are represented in (row, column), the player have to inuut row and column nmber one by one. Both values must pass `.isdigit()` to ensure its data type is number, otherwise "Invalid input" will be shown and ask the player to enter again.
3. **Range Check & occupancy check** – The entered row number and column number should be between 1 to the actual game board size(which is equal to `num_row_column`), and the selected cell should be empty.

All three check need to passed to continue placing the piece into the game board.

Test ID	Input (row, column)	Data Category	Expected	Actual
IV-C-1	("5", "5")	Valid	Piece placed, and continue to next player to place	Passed (As expected)
IV-C-2	("1", "1")	Boundary (Top-left) (Valid)	Piece placed, and continue to next player to place	Passed (As expected)
IV-C-3	("15", "15")	Boundary (Bottom-right)(Valid)	Piece placed, and continue to next player to place	Passed (As expected)
IV-C-4	("", "5")	Empty value (row)	Print "You type nothing!!! Try again, please."	Passed (As expected)
IV-C-5	("5", "")	Empty value (column)	Print "You type nothing!!! Try again, please."	Passed (As expected)
IV-C-6	("", "")	Empty value (row&column)	Print "You type nothing!!! Try again, please."	Passed (As expected)
IV-C-7	("a", "5")	Unmatched data type (row)	Print "Invalid input"	Passed (As expected)
IV-C-8	("5", "b")	Unmatched data type (column)	Print "Invalid input"	Passed (As expected)
IV-C-9	("-1", "5")	Unmatched data type (Negative number)	Print "Invalid input"	Passed (As expected)
IV-C-10	("5.5", "5")	Unmatched data type (Not integer number)	Print "Invalid input"	Passed (As expected)
IV-C-11	(" 5 ", "5")	Whitespace	Ignore whitespaces and get "5" → piece placed.	Passed (As expected)

Test ID	Input (row, column)	Data Category	Expected	Actual
IV-C-12	(" ", "5")	Whitespace	Ignore whitespaces and get empty value → Print "Invalid input"	Passed (As expected)
IV-C-13	("0", "5")	Below range (row)	Print "Invalid input"	Passed (As expected)
IV-C-14	("16", "5")	Out of range (row)	Print "Invalid input"	Passed (As expected)
IV-C-15	("5", "0")	Below range (column)	Print "Invalid input"	Passed (As expected)
IV-C-16	("5", "16")	Out of range (column)	Print "Invalid input"	Passed (As expected)
IV-C-17	("8", "8") then ("8", "8")	Occupied cell	Piece placed at first input → Print "This cell is already occupied" at second input	Passed (As expected)

Test Cases

To test the program, we can simulate different game situations. We can get expected behaviour based on the code logic.

Test ID	Game Situation	Expected	Actual Result
TC-1	Player 1 gets 5 in a row (Horizontal)	"Player 1 wins!"	Passed (As expected)
TC-2	Player 1 gets 5 in a row (Vertical)	"Player 1 wins!"	Passed (As expected)
TC-3	Player 1 gets 5 in a row (Diagonal - From top-left to bottom-right)	"Player 1 wins!"	Passed (As expected)
TC-4	Player 1 gets 5 in a row (Diagonal - From bottom-left to top-right)	"Player 1 wins!"	Passed (As expected)
TC-5	Player 2 gets 5 in a row (Horizontal)	"Player 2 wins!"	Passed (As expected)
TC-7	Board full with no 5-in-a-row	"Board Full. No winner."	Passed (As expected)
TC-8	(PvM Mode) Player 1 (human) get 5 in a row	"Player 1 wins!"	Passed (As expected)

Test ID	Game Situation	Expected	Actual Result
TC-10	(PvM Mode) Player 1 (human) get 4 in a row, machine player move next	The machine player should try to defensive by blocking the winning cell	Passed (As expected)

Unit Test

Unit tests were performed on each subprogram (function) in the code. A complex test can be split into different small task by creating subprogram. Each subprogram have specific actions, they can be called repeatedly without rewriting the code.

Test ID	Subprogram	Description	Setup / Conditions	Expected Return	Actual
UT-1	<code>check_win_v2</code>	Horizontal win (5 in a row)	Row index 6, columns 4–8 = "●" -> Execute <code>check_win_v2(6, 8, "●")</code>	True	Passed
UT-2	<code>check_win_v2</code>	Vertical win (5 in a column)	Column index 4, rows 2–6 = "●" -> Execute <code>check_win_v2(4, 4, "●")</code>	True	Passed
UT-3	<code>check_win_v2</code>	Diagonal win (Top-left > Bottom-right)	Cell (2,2), (3,3), (4,4), (5,5), (6,6) = "●" -> Execute <code>check_win_v2(4, 4, "●")</code>	True	Passed
UT-4	<code>check_win_v2</code>	Diagonal win (Bottom-left > Top-right)	Cell (8,4), (7,5), (6,6), (5,7), (4,8) = "●" -> Execute <code>check_win_v2(6, 6, "●")</code>	True	Passed
UT-5	<code>check_win_v2</code>	Player 2 horizontal win	Row index 3, columns 9–13 = "○" -> Execute <code>check_win_v2(3, 11, "○")</code>	True	Passed
UT-6	<code>is_board_full</code>	Empty board	All cells are empty	False	Passed
UT-7	<code>is_board_full</code>	1 cell occupied	1 cell occupied, remaining empty	False	Passed
UT-8	<code>is_board_full</code>	1 cell empty	1 cell is empty, remaining all occupied	False	Passed
UT-9	<code>is_board_full</code>	Full board	All 225 cells are occupied	True	Passed

System Test

In system test, we will run the game from start to finish instead a part of it. For each step, the expected behaviour can know from the code, and some of step are done in previous testing.

ST-01: [PvP] Player 1 Wins (Horizontal)

Step	Test Step / Input	Expected	Actual	Pass/Fail
1	Launch the game, then select mode "1"	No error message, PvP mode is entered	PvP mode	Pass
2	Player 1: (8,5), (8,6), (8,7), (8,8); Player 2: (2,2), (3,3), (4,4), (5,5)	Piece placed; Two players input coordinate one by one.	As expected	Pass
3	Player 1: (8,9)	As player 1 get 5 in a row first, player 1 should win the game	Board printed with "Player 1 wins!", game ended	Pass

ST-02: [PvP] Player 1 Wins (Vertical)

Step	Test Step / Input	Expected	Actual	Pass/Fail
1	Launch the game, then select mode "1"	No error message, PvP mode is entered	PvP mode	Pass
2	Player 1: (3,7), (4,7), (5,7), (6,7), (7,7); Player 2 randomly place at other position	Piece placed; Two players input coordinate one by one.	As expected	Pass
3	Player 1 will get 5 in a row after placed at (7,7)	Player 1 should win the game	Board printed with "Player 1 wins!", game ended	Pass

ST-03: [PvP] Player 2 Wins (Diagonal)

Step	Test Step / Input	Expected	Actual	Pass/Fail
1	Launch the game, then select mode "1"	No error message, PvP mode is entered	PvP mode	Pass
2	Player 2: (3,3), (4,4), (5,5), (6,6), (7,7); Player 1 place elsewhere	Piece placed; Two players input coordinate one by one.	As expected	Pass
3	Player 2 complete 5 in diagonal after placed at (7,7)	Player 2 should win the game	Board printed with "Player 2 wins!", game ended	Pass

ST-04: [PvP] Draw (Board Full)

Step	Test Step / Input	Expected	Actual	Pass/Fail
1	Launch the game, then select mode "1"	No error message, PvP mode is entered	PvP mode	Pass
2	Fill whole board with no one get 5 in-a-row	Piece placed. And no board full message show during filling the whole board	As expected	Pass
3	After the last piece filled the board	is_board_full should return board full, and the game have no winner	"Board Full. No winner.", draw detected, round is end	Pass

ST-05: [PvM] Human Player Wins

Step	Test Step / Input	Expected	Actual	Pass/Fail
1	Launch the game, then select mode "2"	No error message, PvM mode is entered	PvM mode	Pass
2	Player 1 builds a 5 in horizontal while the machine defense	Machine player should 3x3 near the last human player's move, and follow machine logic to.	Machine responed and piece placed.	Pass
3	Player 1 (human) places the 5th piece	Human player should win the game	Win detected: "Player 1 wins!", game ended.	Pass

ST-06: [Input Validation] Invalid input

Step	Test Step / Input	Expected	Actual	Pass/Fail
1	Mode: "", then "3", then "1"	"Invalid input" should shown at first two entry, then enter PvP mode after "1" entered	As expected. User is asked to select again after invaild input.	Pass
2	Enter empty value in row or column	"You type nothing!!! Try again, please."	Message shown	Pass
3	Enter a letter instead of a number in row or column	"Invalid input"	Message shown	Pass
4	Enter a number that exceed the board size in row or column	"Invalid input"	Message shown	Pass

Step	Test Step / Input	Expected	Actual	Pass/Fail
5	Enter same coordinate twice time	"This cell is already occupied" should show at the second time	Message shown	Pass

User Acceptance Test

UAT was conducted with 5 testers who played this game (including PvP and PvM modes) and willing to fill a form to provide feedback.

Their feedbacks are collected and separated into following aspects:

Positive

1. Colour-coded and clearly formatted game board make the game easy to use
2. The program run normally without run-time error. And dependencies are easy to install by following intructions.
3. Clean the terminal before printing the game board make the game experinece better.

Issues

1. Player cannot choose to play again, the program exits after every game, and have to run the code again mannually.
2. The performance of machine player in PvM mode is poor. It only focused at the last player's move.
3. The game only show "invalid input" message without any reasons.
4. The program will run normally even the game board size is entered a negative number

Suggestions

1. Add "Play-again" feature, allowing player choose to start again after the end of one game. Score tracking can also be added to count total mark for multiple round.
2. Make the error messages clearer by adding reasons.
3. Refactor the machine logic to make it smarter or connect to AI services to make choices. Predict the players' move and evaluate the whole board.

Debugging & Modifications

After testing and collect UAT feedback, the program code is updated to fix bugs and add new feature to improve game experience.

Bug #1 - Invalid num_row_column

The size of the game board is equal to the value of variable `num_row_column` in the code. During UAT, we found that if a negative number is entered, the game start with a very small game board, and no vaild coordinate. To avoid this situation, a validation check should be added right after the variable definition and before the board initialisation. If the game board size is entered smaller than 1, the program should return error and prompt the player to change the game board size to a valid one.

Updated code:

```
num_row_column = 15 # Board Size

# Board Size Validation - num_row_column cannot be less than 1
if num_row_column < 1:
    raise ValueError("Invalid board size: num_row_column must be at least 1.")
```

Modification #1 - Play-again feature & Score tracking

```
Player 1 wins!
Score - Player 1: 1 | Player 2: 0 | Draw: 0
Play again? (y/n): n
Thanks for playing! Final score - Player 1: 1 | Player 2: 0 | Draw: 0
```

After UAT, some tester suggest add a play-again feature, so this modification is made. After updating the code, the game will no longer exits after one round is finished. With this feature, player can start a new round by confirming "y" without running the code again manually.

Considering that players may use multiple rounds to determine the winner a score tracking feature is added. The program will record the score of each player, if one of them win a round, the score will add 1, and no mark for anyone if draw.

In task 1, the main loop ended the whole program by calling `break` after one player win or the game board is full, this limit the program can only play one round. After updating, the main loop is restructured to use a flag `round_ended` instead of `break`.

```
# Main
while True:
    # Start a new round
    board.clear()
    board_init()
    round_ended = False

    while not round_ended:
        .....
        if not round_ended:
            turn = 1 - turn # Player round switch

    # End of round - show the total score
    cprint(f"Score - Player 1: {score_player_1} | Player 2: {score_player_2} | Draw: {score_draw}", "cyan", attrs=["bold"])
```

For the score tracking, three score counters variables is added.

```
# Score tracking
score_player_1 = 0
score_player_2 = 0
score_draw = 0
```

When win is detected, the winner player's score will add 1. For example, when the program detected Player 1 win:

```
if check_win_v2(board, selected_row - 1, selected_column - 1,
player_1_indicator):
    print("\033c", end="")
    board_print(board)
    cprint("Player 1 wins!", "green", attrs=["bold"])
    score_player_1 += 1      # Add 1 mark for player 1
    round_ended = True
```

After each round ended, the player will be asked whether to play a new round. If the player confirm to start a new round, the outer loop continues and the board is cleared. If the player choose to end the whole game (replying "n"), the final score for both player will be printed and the program exits.

```
# Play-again
while True:
    play_again = input("Play again? (y/n): ").strip().lower() # Remove
    whitespace and convert to lowercase letter
    if play_again == "y":
        break # Start a new round (outer loop continues)
    elif play_again == "n":
        cprint(f"Thanks for playing! Final score - Player 1:
{score_player_1} | Player 2: {score_player_2} | Draw: {score_draw}",
"cyan", attrs=["bold"])
        exit() # END OF WHOLE PROGRAM
    else:
        cprint("Invalid input: please enter 'y' to play again or 'n' to
quit.", "red", attrs=["bold"])
```

Modification #2 - Error messages with reasons

In task 1, invalid inputs was rejected with "Invalid input" message (For empty value: "You type nothing!!! Try again, please."), which does not tell the player why the input was rejected.

All error messages is now include the reason, the player knows what exactly they entered wrong.

The message at mode selection now tell the player the valid options that they supposed to input:

```

else:
    cprint("Invalid input: please enter 1 (PvP) or 2 (PvM).", "red", attrs=
["bold"])

```

For coordinate input, the validation is split into steps, so that each case report its own specific reason. The specific invalid data (row or column) is shown and the valid range also will shown.

```

while True:
    selected_row = input(colored("Enter the row: ", "blue", attrs=
["bold"]))
    selected_column = input(colored("Enter the column: ", "blue", attrs=
["bold"]))
    if selected_row == '' or selected_column == '': # Input nothing
        cprint("You type nothing!!! Row and column cannot be empty.",
"red", attrs=["bold"])
        continue # Jump to next loop: Input again
    selected_row = selected_row.strip() # Remove the whitespace
    selected_column = selected_column.strip()
    if not (selected_row.isdigit() and selected_column.isdigit()): # Check
the strings only contain digit number
        if not selected_row.isdigit() and not selected_column.isdigit():
            cprint(f"Invalid input: both row '{selected_row}' and column
'{selected_column}' are not numbers (1-{num_row_column}).", "red", attrs=
["bold"])
        elif not selected_row.isdigit():
            cprint(f"Invalid input: row '{selected_row}' is not a number
(1-{num_row_column}).", "red", attrs=["bold"])
        else:
            cprint(f"Invalid input: column '{selected_column}' is not a
number (1-{num_row_column}).", "red", attrs=["bold"])
        continue
    selected_row = int(selected_row)
    selected_column = int(selected_column)
    break

```

The out-of-range and occupied-cell messages also point out exactly which coordinate is invalid:

```

if selected_row > 0 and selected_row <= num_row_column and selected_column
> 0 and selected_column <= num_row_column: # Data validation - within the
board size
    if board[selected_row - 1][selected_column - 1] ==
empty_cell_indicator: # Data validation - EMPTY CELL
        .....
    else:
        cprint(f"This cell ({selected_row},{selected_column}) is already
occupied. Please choose an empty cell.", "red", attrs=["bold"])
else:
    if selected_row < 1 or selected_row > num_row_column:

```

```
cprint(f"Invalid input: row {selected_row} is out of range (1-  
{num_row_column}).", "red", attrs=["bold"])  
else:  
    cprint(f"Invalid input: column {selected_column} is out of range  
(1-{num_row_column}).", "red", attrs=["bold"])
```

Algorithm Optimisation

Current machine logic

In PvM mode, the machine player uses a simple decision logic. It first collects all empty cells within the 3x3 area around the human player's last move, then decides the move in this order:

1. **Defend** - The machine first search all empty cell nearby the human player's last move. If the human player can get 5-in-a-row by placing on one of these cells, the machine will place its piece to that cell to block human player.
2. **Attack** - If the machine player can complete its own 5-in-a-row, the machine will place its piece at that cell.
3. **Random** - If neither defend and attack applies, the machine player will select an empty cell randomly nearby, or a random cell on the game board if there is no empty cell nearby.

This logic is simple and fast, but it limits the machine will only search around the last move and cannot predict next move, so the human player can beat the machine easily.

Improvements made

- **Board size validation** - A data validation is added to reject invalid board size (e.g. a negative number) by rising error to let player change it, instead of continuing the game with a broken board.
- **Play-again loop and score tracking** - The main loop is restructured (Mentioned in Modif#1) to support multiple rounds and record the mark.
- **Clearer input validation** - The data validation logic in coordinate input is conserved, but each error message now tells the player what exactly they entered wrong, so invalid input is handled more clearly.

Possible further improvements

- **Reduce win checking time** - The subprogram `check_win_v2` can skip directions where 5 in a row is impossible (e.g. the piece is at the edge of the board).
- **Score the whole board** - (For PvM mode) Instead of only searching the cells near the last move or making random select, the machine can evaluate empty cells on the board and give each one a score based on how many piece the cell can connect in a row (maybe can for both the human and the machine?). The cell with the highest score should be chosen after evaluating. This method may make the machine smarter, but the machine will consider the whole board instead nearby, this may make the code complex and cost more resources.
- **Connect to external public AI service** - Instead of following a fixed logic, the program can send the current game board to an external AI service and let the AI to select. To integrate the AI, we

might consider working with both OpenAI- and Anthropic-compatible APIs, the base URL and key should be configured by the user. But when the program has no internet connection or the API unavailable, the program should return to follow the fixed logic that we wrote in the code.

Conclusion

In this report, the Gomoku game was tested and evaluated by different data. Based on the code, the program design was analysed for its strengths and weaknesses. A set of tests (including input validation, unit tests, system tests and user acceptance test) was designed and carried out, and most of the tests passed as expected. Based on the testing result and UAT feedback, bugs was fixed, and improved by adding new feature and refactoring. For further development, some algorithm optimisation ideas was discussed. Although there is still room for improvement, the program now run stably and as expected. Compared to task 1, the optimized program in task 2 improved player experience.

Program Full Code

The source code of the program is also available in [Github](#)

```
# Gomoku Game
# 25-26 ICT SBA Guided Task 2 (Modified from Task 1)
# Library required: random, time, tabulate, termcolor

# Dependencies
try:
    import random, time
    from tabulate import tabulate # type: ignore
    from termcolor import colored, cprint # type: ignore
except ModuleNotFoundError as e:
    raise ModuleNotFoundError(
        "Missing dependency. Please follow instructions from README.md"
    ) from e

# Variable
turn = 0
empty_cell_indicator = " "
player_1_indicator = "●"
player_2_indicator = "○"
num_row_column = 15 # Board Size

# Board Size Validation - num_row_column cannot be less than 1
if num_row_column < 1:
    raise ValueError("Invalid board size: num_row_column must be at least 1.")

board = []

# Score tracking
score_player_1 = 0
score_player_2 = 0
```

```

score_draw = 0

# Func
# Board initializing
def board_init():
    turn = 0
    for i in range(num_row_column):
        board.append([])
        for r in range(num_row_column):
            board[i].append(empty_cell_indicator)

## Board Print
def board_print(board):
    headers = [" "]
    for i in range(1, num_row_column + 1):
        headers.append(str(i))

    rows = []
    for i in range(num_row_column):
        row = [str(i + 1)]
        for cell in board[i]:
            row.append(cell)
        rows.append(row)
    print(
        tabulate(
            rows,
            headers=headers,
            tablefmt="rounded_grid",
            stralign="center",
            numalign="center",
        )
    )
    print()

## Check Win (V2)
def check_win_v2(board, row, col, indicator):
    directions = [
        (0, 1), # Horizontal
        (1, 0), # Vertical
        (1, 1), # Right-down diagonal
        (1, -1) # Left-down diagonal
    ]
    for dr, dc in directions:
        count = 1 # Initial count

        # Positive
        r, c = row + dr, col + dc
        while 0 <= r < len(board) and 0 <= c < len(board[0]) and board[r]
[c] == indicator:
            count += 1
            r += dr
            c += dc

        # Negative
        r, c = row - dr, col - dc

```

```

        while 0 <= r < len(board) and 0 <= c < len(board[0]) and board[r]
[c] == indicator:
            count += 1
            r -= dr
            c -= dc

        # Satisfy win condition: 5 in a row (or more)
        if count >= 5:
            return True

    return False

# Board Full Check
def is_board_full(board):
    for row in board:
        if empty_cell_indicator in row:
            return False
    return True

# Mode Selection
print("Please select your preferred mode:")
print(" 1. Player vs Player (PvP) \n 2. Player vs Machine (PvM)")
while True:
    choice = input("Select mode (1/2): ")
    if choice == "1":
        game_mode = "PvP"
        break
    elif choice == "2":
        game_mode = "PvM"
        break
    else:
        cprint("Invalid input: please enter 1 (PvP) or 2 (PvM).", "red",
attrs=["bold"])
        cprint(f"You have selected {game_mode} mode", "green", attrs=["bold"])
        time.sleep(1.5)

# Main
while True:
    # Start a new round
    board.clear()
    board_init()
    round_ended = False

    while not round_ended:
        print("\033c", end="") # Clean console (ANSI Escape Codes)

        # Player turn Noti
        if turn == 0:
            cprint("Player 1 turn \n", "green", attrs=["bold"])
            cprint("Your piece is: " + player_1_indicator, "magenta",
attrs=["bold"])
        else:
            cprint("Player 2 turn \n", "red", attrs=["bold"])
            cprint("Your piece is: " + player_2_indicator, "magenta",

```

```

attrs=["bold"])
    board_print(board)

    # Player input & Data validation
    while True:
        if turn == 1 and game_mode == "PvM":            # PvM mode -
machine turn
            # Machine turn logic
            cprint("Machine is thinking...", "yellow", attrs=["bold"])

            last_player_row_index = selected_row - 1
            last_player_column_index = selected_column - 1
            nearby_empty_cells = []

            # 1. Search nearby empty cells
            for temp_row in range(-1, 2):
                for temp_column in range(-1, 2):
                    current_check_row = last_player_row_index +
temp_row
                    current_check_column = last_player_column_index +
temp_column
                    if 0 <= current_check_row < num_row_column:
                        if 0 <= current_check_column < num_row_column:
                            if board[current_check_row]
[current_check_column] == empty_cell_indicator:
nearby_empty_cells.append((current_check_row, current_check_column))

                    pvm_machine_selected_move = None

            # 2. Defense First
            for empty_row, empty_column in nearby_empty_cells:
                if check_win_v2(board, empty_row, empty_column,
player_1_indicator):
                    pvm_machine_selected_move = (empty_row,
empty_column)
                    break

            # 3. Attack if no immediate threat
            if not pvm_machine_selected_move:
                for empty_row, empty_column in nearby_empty_cells:
                    if check_win_v2(board, empty_row, empty_column,
player_2_indicator):
                        pvm_machine_selected_move = (empty_row,
empty_column)
                        break

            # 4. FALLBACK: Random Move
            if not pvm_machine_selected_move:
                if len(nearby_empty_cells) > 0:
                    pvm_machine_selected_move =
random.choice(nearby_empty_cells) # Random select one nearby cell
                else: # IF: No empty cell nearby
                    all_empty_on_board = []

```



```

        for row_index in range(num_row_column): # Find all
empty cells in the game board
            for column_index in range(num_row_column):
                if board[row_index][column_index] ==
empty_cell_indicator:
                    all_empty_on_board.append((row_index,
column_index))

                    pvm_machine_selected_move =
random.choice(all_empty_on_board) # Random select one empty cell in whole
game board

                    # 5. Apply machine selected move
                    board[pvm_machine_selected_move[0]]
[pvm_machine_selected_move[1]] = player_2_indicator
                    selected_row = pvm_machine_selected_move[0] + 1
                    selected_column = pvm_machine_selected_move[1] + 1

                    time.sleep(1.5) # Thinking time, just more realistic

                    cprint(f"Machine: I selected ({selected_row},
{selected_column}).", "blue")

                    time.sleep(1.5) # Machine selected result display

                    break # Exit input loop, continue to Check win part

    else: # PvP mode & PvM mode (Human player)
        # Human turn logic (Modified: clearer error messages)
        while True:
            selected_row = input(colored("Enter the row: ", "blue",
attrs=["bold"]))
            selected_column = input(colored("Enter the column: ",
"blue", attrs=["bold"]))
            if selected_row == '' or selected_column == '': #
Input nothing
                cprint("You type nothing!!! Row and column cannot
be empty.", "red", attrs=["bold"])
                continue # Jump to next loop: Input again
            selected_row = selected_row.strip() # Remove the
whitespace
            selected_column = selected_column.strip()
            if not (selected_row.isdigit() and
selected_column.isdigit()): # Check the strings only contain digit number
                if not selected_row.isdigit() and not
selected_column.isdigit():
                    cprint(f"Invalid input: both row
'{selected_row}' and column '{selected_column}' are not numbers (1-
{num_row_column}).", "red", attrs=["bold"])
                elif not selected_row.isdigit():
                    cprint(f"Invalid input: row '{selected_row}' is
not a number (1-{num_row_column}).", "red", attrs=["bold"])
                else:
                    cprint(f"Invalid input: column
'{selected_column}' is not a number (1-{num_row_column}).", "red", attrs=

```

```

["bold"]))
        continue
        selected_row = int(selected_row)
        selected_column = int(selected_column)
        break

        if selected_row > 0 and selected_row <= num_row_column and
selected_column > 0 and selected_column <= num_row_column: # Data
validation - within the board size
        if board[selected_row - 1][selected_column - 1] ==
empty_cell_indicator: # Data validation - EMPTY CELL
        if turn == 0:
            board[selected_row - 1][selected_column - 1] =
player_1_indicator
        else:
            board[selected_row - 1][selected_column - 1] =
player_2_indicator
        break # Exit inner loop - input valid & finish
replacing the cell
    else:
        cprint(f"This cell ({selected_row},
{selected_column}) is already occupied. Please choose an empty cell.",
"red", attrs=["bold"]) # Repeat loop - Input valid & Cell occupied
    else:
        if selected_row < 1 or selected_row > num_row_column:
            cprint(f"Invalid input: row {selected_row} is out
of range (1-{num_row_column}).", "red", attrs=["bold"])
        else:
            cprint(f"Invalid input: column {selected_column} is
out of range (1-{num_row_column}).", "red", attrs=["bold"])

# Check win
if turn == 0:
    if check_win_v2(board, selected_row - 1, selected_column - 1,
player_1_indicator):
        print("\033c", end="")
        board_print(board)
        cprint("Player 1 wins!", "green", attrs=["bold"])
        score_player_1 += 1
        round_ended = True
    else:
        if check_win_v2(board, selected_row - 1, selected_column - 1,
player_2_indicator):
            print("\033c", end="")
            board_print(board)
            cprint("Player 2 wins!", "green", attrs=["bold"])
            score_player_2 += 1
            round_ended = True

# Check the board whether is full or not.
if not round_ended and is_board_full(board):
    print("\033c", end="")
    board_print(board)
    cprint("Board Full. No winner.", "yellow", attrs=["bold"])

```

```
        score_draw += 1
        round_ended = True

    if not round_ended:
        turn = 1 - turn # Player round switch

# End of round - show the total score
cprint(f"Score - Player 1: {score_player_1} | Player 2:
{score_player_2} | Draw: {score_draw}", "cyan", attrs=["bold"])

# Play-again
while True:
    play_again = input("Play again? (y/n): ").strip().lower() # Remove
whitespace and convert to lowercase letter
    if play_again == "y":
        break # Start a new round (outer loop continues)
    elif play_again == "n":
        cprint(f"Thanks for playing! Final score - Player 1:
{score_player_1} | Player 2: {score_player_2} | Draw: {score_draw}",
"cyan", attrs=["bold"])
        exit() # END OF WHOLE PROGRAM
    else:
        cprint("Invalid input: please enter 'y' to play again or 'n' to
quit.", "red", attrs=["bold"])
```

- END OF THIS REPORT -

Task 1 | [Github: a1e0xye-school-project/SBA_Guided-Task-1](#)

Task 2 | [Github: a1e0xye-school-project/SBA_Guided-Task-2](#)

By Alex Ye

2026/7-2026/8

Thank you!